Type Systems

CG

November 7, 2026

Contents

1 Types **2**

1.1 Kinds of typing . . . 3
1.2 Basic, primitive and composite types . . . 3
1.3 Type constructors . . . 4
1.4 Static and dynamic type checking . . . 5
1.5 Newer types . . . 6
1.6 Well-typed programs cannot go wrong . . . 7
1.7 Exercises . . . 7

2 Typing rules and LETREC **8**

2.1 Typing rules . . . 8
2.2 The language LETREC . . . 8
2.3 Typing rules of LETREC . . . 10
2.4 The function type-of . . . 11
2.5 Type checking and type inference . . . 13
2.6 Type environments . . . 13
2.7 Exercises . . . 14

3 Lambda calculus **14**

3.1 Syntax . . . 14
3.2 Reduction . . . 15
3.3 Church encodings . . . 15
3.4 Recursion . . . 16
3.5 Exercises . . . 16

4 Simply typed lambda calculus **16**

4.1 Types and terms . . . 16
4.2 The typing judgment . . . 17

4.3 Capture-avoiding substitution . . . 17
4.4 Strong normalization . . . 17
4.5 Propositions as types . . . 17
4.6 Exercises . . . 18

5 Type inference **18**
5.1 Rules with type variables . . . 19
5.2 The algorithm . . . 20
5.3 Principal types and Algorithm W . . . 24
5.4 Exercises . . . 25

6 Polymorphism **25**
6.1 Parametric polymorphism . . . 25
6.2 Ad hoc polymorphism . . . 26
6.3 Higher-rank polymorphism . . . 27
6.4 Exercises . . . 27

7 Subtyping **27**
7.1 Subsumption . . . 27
7.2 Variance . . . 28
7.3 Nominal and structural subtyping . . . 29
7.4 Exercises . . . 29

8 Recursive types **29**
8.1 μ-types . . . 30
8.2 Algebraic data types . . . 30
8.3 Exercises . . . 31

9 Type safety **31**
9.1 Lemmas . . . 32
9.2 Proof strategy . . . 32
9.3 Exercises . . . 32

1 Types

Definition 1.1. A **type** is a specification of the operations, the data structure, the external interface and the internal representation of the elements of a program, its values.

A type system gives a programming language a logic: rules that describe which transformations are allowed on which data. This block follows Watt [3, Ch. 2, 8], Friedman and Wand [1, Ch. 4]

and Lee [2, Sec. 1.5].

1.1 Kinds of typing

The lecture compares three ways in which languages handle types (Table 1).

Typing	Values	When operations are checked
Strong	each value has one type	statically, while the program is built
Inferred	each value has one type	statically, from types given by the programmer or deduced from the use of each value
Dynamic	a value can have different types at different times; at run time it is stored as a pair (value, type tag)	at run time

Table 1: Kinds of typing.

Discussions about type systems usually turn around four questions, and a language answers each one independently:

- static or dynamic: are types checked before the program runs or while it runs?
- strong or weak: may the language reinterpret a value as another type without an explicit conversion? In Python `"3" + 3` is an error; in JavaScript it gives `"33"`.
- explicit or inferred: does the programmer write the types, or does the compiler deduce them?
- nominal or structural: are two types compatible because of their names, or because of their shape?

1.2 Basic, primitive and composite types

Following Martin-Löf, three types are the basis of all others. The empty type $\bot$ has no elements; it is used to express negation, $\neg A ::= A \to \bot$. The unit type has a single element, written `()`, `unit` or `null` depending on the language. The two type has two values, `true` and `false`. All other types have infinitely many elements.

Languages provide **primitive types** (integers, booleans, characters, pointers) and let the programmer define **composite types** (functions, lists, objects, arrays). Primitive types are defined by the range of values they represent: `int` is $-2147483648..2147483647$ and `byte` is $-128..127$ in Java, and Ada lets the programmer declare new ranges:

```
type Population is range 0..1e10
type Row is range a..n
```

Composite types are built from simpler types by product, sum, mapping or recursion. The lecture's examples are an Ada record, which is a product of its fields, and a class hierarchy, which behaves as a disjoint union of the classes that extend the base class:

```
type Month is (jan, feb, ..., dec)
type Day is range 1..31
type Date is record
  m: Month;
  d: Day;
end record;
```

1.3 Type constructors

Table 2 lists the constructors of the lecture. The names Σ and Π come from Martin-Löf's type theory, where they generalize the cartesian product and the function space.

Constructor	Notation	Meaning
Function	$T_1 \to T_2$	functions from T_1 to T_2
Σ-type	$\sum_{i=1}^{n} T_i = \{(v_1, \ldots, v_n) \mid v_i : T_i\}$	tuples of values of the given types; structures and records
Π-type	$\prod_{v:T_1} T_2$	functions whose result type may depend on the input value
Equality	$exp_1 : T = exp_2 : T$	the type of proofs that two expressions reduce to the same one, as $2 + 2 = 2 * 2$
Inductive	$0 : \mathbb{N},\ \ s : \mathbb{N} \to \mathbb{N}$	self-referential types built from constants and constructor functions

Table 2: Type constructors.

Function types and currying. The type of a function goes from the types of its parameters to the type of its result. A function of several parameters can always be seen as a function of one parameter that returns a function: a function of two integers that returns an integer has type $int \times int \to int$, or, curried, $int \to (int \to int)$. This is the currying property, and the arrow associates to the right, so the parentheses are usually omitted: $int \to int \to int$.

Σ and Π. A Σ-type is the type of tuples: if $T_1, \ldots, T_n$ are types, $\sum T_i$ contains the tuples $(v_1, \ldots, v_n)$ with $v_i : T_i$. A Π-type is the type of functions, where the type of the output can depend on the input value: for each element v of T_1 the function returns an element of T_2, and T_2 may mention v. When T_2 does not depend on v, $\prod_{v:T_1} T_2$ is just $T_1 \to T_2$. (In programming languages the word "sum type" usually means a disjoint union, as `Either` in Haskell; in type theory the disjoint union is written $A + B$, and Σ is a generalized product.)

Inductive types. An inductive type is defined by constant elements and constructor functions. The natural numbers of Peano have the constant $0 : \mathbb{N}$ and the successor $s : \mathbb{N} \to \mathbb{N}$, and the elements of the type are $0, s0, ss0, sss0, \ldots$. Lists and trees are inductive types too; Section 8 returns to them.

1.4 Static and dynamic type checking

Static typing. Every entity of the program has a type at compile time, and the type checker verifies that the types are consistent before the program runs. Its advantages: it catches errors early, it gives guarantees about the programs, and the hardware can sometimes use the types directly (an integer known at compile time needs no tag). Its cost: the programmer has to write the types of the values, which type inference (Section 5) reduces.

Dynamic typing. The same checks are made while the program runs. For that, every value carries its type, as a pair (value, type). The lecture uses Prolog to show it. A predicate `list_length([_ | L], N)` computes the length of a list; called with a list it gives $N = 4$, and called as `list_length(9, N)` the tags say that an `Int` is not a `List`, and the error is found only when the call runs. The same idea in Scheme, with explicit tags:

```
(define (tagged value type) (cons value type))
(define (value-of v) (car v))
(define (type-of v) (cdr v))

(define (check-type v expected)
  (if (not (eq? (type-of v) expected))
      (error "type error:" (type-of v) 'is-not expected)))

(define (list-length v)
  (check-type v 'list)
  (let loop ((l (value-of v)) (n 0))
    (if (null? l)
        (tagged n 'int)
        (loop (cdr l) (+ n 1)))))
```

The test calls `list-length` with a value tagged `int` and checks that the error says `int is-not list`. Dynamic typing is more flexible when programming, but execution costs more (the checks are repeated every time an operation runs), the type information takes storage, and the lecture adds that it promotes careless programming: an error in a branch that the tests never execute stays hidden. Figure 1 contrasts when each approach finds the error.

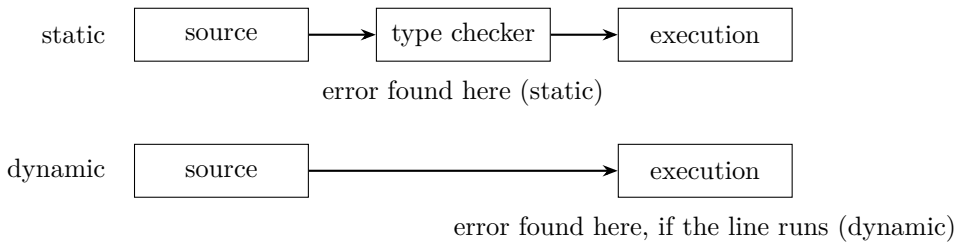


Figure 1: Where a type error is found.

Duck typing. Types do not have to be structural: the type of an expression can also be determined by its behavior. "If it talks like a duck and walks like a duck, then it is a duck." The lecture shows a Go interface `Quacker` that any type with a method `Quack()` satisfies without declaring it. In Python the check happens at run time:

```
class Duck:
    def quack(self):
        return "Quack!"

class Robot:
    def quack(self):
        return "Beep quack"

class Dog:
    def bark(self):
        return "Woof"

def make_it_quack(thing):
    return thing.quack()
```

`make_it_quack` accepts a `Duck` and a `Robot`, which have nothing in common except the method. With a `Dog` the error appears when the call runs.

1.5 Newer types

Modern type systems add types that make common errors impossible.

Option types. An option type (or Maybe type) T? holds a value of T or nothing. It is used for function results that may have no meaningful value. The lecture writes it in Scala, with `Some(42)` and `None`; in Swift `Int?` is the same type, with `nil` for nothing:

```
func showValue(_ opt: Int?) -> String {
    if let x = opt {
        return "The value is: \(x)"
    }
    return "No value"
}
```

To use the value the program has to open the option (`if let`), so the case without a value cannot be forgotten.

Nullable types. A `null` value makes operations unsafe: any expression of a reference type may suddenly be `null`. A nullable type system separates T (always a valid reference) from T? (a reference or `null`), as in Figure 2, and the compiler checks that a T? is never used as a T without a check.

```
func greeting(_ name: String?) -> String {
    guard let unwrapped = name else {
        return "No name provided."
    }
    return "Hello, \(unwrapped)!"
}

func greetUser(username: String) -> String {
    return "Welcome, \(username)!"
}
```

`greetUser` takes a `String`, not a `String?`, so the call `greetUser(username: nil)` is rejected at compile time with `'nil' is not compatible with expected argument type 'String'`.

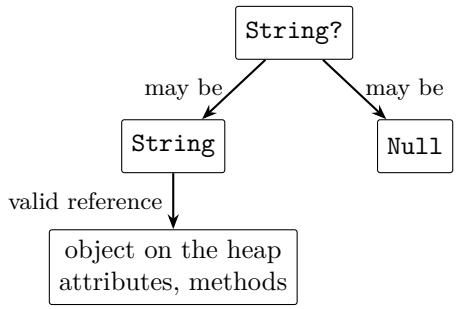


Figure 2: A nullable type is a valid reference or null.

Owned types. In Rust every value is attached to one variable, its owner, and has exactly one owner at a time. Assigning a value or passing it as a parameter moves it to a new owner, and the old variable can no longer be used. A value can also be borrowed (`&s`): passed temporarily to a function and returned to its owner. In the lecture's example, `print_string(s)` moves the string `s` into the function, so a second `print_string(s)` is a compile error; `print_string(&s)` only borrows it and can be repeated. Figure 3 shows what moves: the variable holds a pointer, a length and a capacity, and the characters live on the heap. Moving copies the three fields to the new owner and invalidates the old one, so the heap memory always has a single owner that frees it.

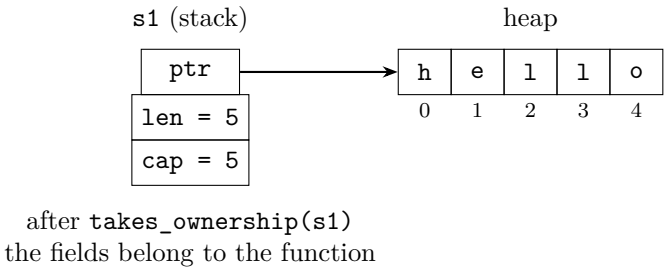


Figure 3: A Rust `String`: the owner holds pointer, length and capacity; the text is on the heap.

1.6 Well-typed programs cannot go wrong

Robin Milner summarized the goal of static type systems as "well-typed programs cannot go wrong". Going wrong has a precise meaning: reaching a state where no rule applies, such as applying the number 3 as a function. The claim is proved as two theorems, progress and preservation, in Section 9. The sections in between build what the proof needs: typing rules and the language LETREC (Section 2), the lambda calculus and its typed version, type inference, polymorphism, subtyping and recursive types.

1.7 Exercises

Exercise 1.2. Classify Python, Java and Swift by the four questions of this section (static or dynamic, strong or weak, explicit or inferred, nominal or structural).

Solution. Python: dynamic, strong, no declarations (types are checked at run time), and duck typing, which is structural by behavior. Java: static, strong, explicit (with local inference through `var`), nominal: a class is a subtype of an interface only if it declares `implements`. Swift: static, strong, inferred where possible, nominal for classes and protocols (a type conforms to a protocol by declaring it).

Exercise 1.3. Give the type of a curried function `add` of two integers, and the type of `add 3`.

Solution. $add : int \to int \to int$, which is $int \to (int \to int)$. Applying it to 3 gives a function $int \to int$ that adds 3.

Exercise 1.4. Write the natural number 3 and the list $[1, 2]$ as elements of inductive types.

Solution. $3 = s(s(s(0)))$. With the constructors $nil : List$ and $cons : \mathbb{N} \to List \to List$, $[1, 2] = cons(s(0), cons(s(s(0)), nil))$.

Exercise 1.5. Does "well-typed programs cannot go wrong" mean that well-typed programs always terminate?

Solution. No. A well-typed program never gets stuck, but it may run forever: in LETREC, `letrec int f (x : int) = (f x) in (f 0)` is well typed and loops.

2 Typing rules and LETREC

2.1 Typing rules

Typing rules define which transformations are allowed on the values of a program, through the functions that apply to each type. A rule has premises above a line and a conclusion below it. The basic statement is a **typing judgment**

$$\Gamma \vdash exp : T$$

read "in the typing context Γ, the expression exp has type T". The context Γ is a map from variables to types: the types of the variables in scope. The lecture's first rule is functional abstraction:

$$\frac{\Gamma, x : T_1 \vdash exp : T_2}{\Gamma \vdash \lambda x : T_1.\, exp : T_1 \to T_2} \text{ (Abs)}$$

If, in the context Γ extended with x of type T_1, the body exp has type T_2, then the function that takes x and returns exp has the function type $T_1 \to T_2$.

2.2 The language LETREC

The lecture uses a small language, LETREC, from Friedman and Wand [1, Ch. 4]:

- it has numbers and boolean values;
- variables are introduced with `let`;

- the only arithmetic expression is the difference `-(e1, e2)`;
- there are `if` expressions and the test `zero?`;
- new functions can be defined (`proc`, and `letrec` for recursive ones) and applied (`(f e)`).

The grammar of the lecture is

```
Expression ::= letrec Identifier (Identifier) = Expression in Expression
             | (Expression Expression)
             | proc (Identifier) Expression
             | if Bool-exp then Expression else Expression
             | num | bool
             | - (Expression, Expression)
Bool-exp   ::= Expression Operator Expression | Identifier (Expression)
Operator   ::= < | > | <> | || | && | ==
```

and a program that doubles a number by recursion:

```
letrec double(x)
  = if zero?(x) then 0 else -((double -(x,1)), -2)
in (double 6)
```

Subtracting -2 adds 2, so `(double 6)` is 12. The typed version of the language adds types to the parameters and to the result of `letrec`:

```
Expression ::= letrec Type Identifier (Identifier : Type) = Expression in Expression
             | proc (Identifier : Type) Expression
             | ...
Type       ::= int | boolean
```

```
letrec
  int double (x : int) = if zero?(x) then 0 else -((double -(x,1)), -2)
in double
```

Typed declarations can come before the variable (`T var`, as in C) or after it (`var : T`, as here). A function can be typed by its parameters and result, $T_{res}\ foo(T_1\ p_1, \ldots, T_n\ p_n)$ or $foo(p_1 : T_1, \ldots, p_n : T_n) : T_{res}$, or with its full type $foo : T_1 \to T_2 \to \cdots \to T_{res}$; the first two forms mean the third.

In Rascal, the syntax with optional types (used also for inference in Section 5) and the abstract syntax are:

```
layout Layout = [\ \t\n]* !>> [\ \t\n];
keyword Reserved = "let" | "in" | "if" | "then" | "else" | "proc" | "letrec"
                 | "true" | "false" | "int" | "bool";
lexical Id  = ([a-z][a-zA-Z0-9]* !>> [a-zA-Z0-9]) \ Reserved;
lexical Num = "-"? [0-9]+ !>> [0-9];

start syntax Ex
  = Num
  | "true" | "false"
  | Id
  | "-" "(" Ex "," Ex ")"
  | "zero?" "(" Ex ")"
  | "if" Ex "then" Ex "else" Ex
```

```
  | "let" Id "=" Ex "in" Ex
  | "proc" "(" Id OptTy ")" Ex
  | "(" Ex Ex ")"
  | "letrec" OptTy Id "(" Id OptTy ")" "=" Ex "in" Ex
  ;

syntax OptTy = ":" Ty | Ty | ;
syntax Ty    = "int" | "bool" | "(" Ty "-\>" Ty ")";
```

```
data Exp
  = const(int n)
  | boolean(bool b)
  | var(str x)
  | diff(Exp lhs, Exp rhs)
  | zero(Exp arg)
  | ifExp(Exp cond, Exp thenExp, Exp elseExp)
  | let(str x, Exp val, Exp body)
  | proc(str x, OType xType, Exp body)
  | call(Exp rator, Exp rand)
  | letrec(OType resType, str f, str x, OType xType, Exp fBody, Exp body)
  ;

data OType = noType() | aType(Type t);

data Type = intType() | boolType() | procType(Type arg, Type res) | tvar(int n);
```

In the concrete syntax a function type is written `(int -> int)`. In the abstract syntax it is `procType(intType(), intType())`.

2.3 Typing rules of LETREC

$$\frac{}{\Gamma \vdash n : \mathit{int}}\ (\textsc{Const}) \qquad \frac{(x : T) \in \Gamma}{\Gamma \vdash x : T}\ (\textsc{Var}) \qquad \frac{\Gamma \vdash e : \mathit{int}}{\Gamma \vdash \texttt{zero?}(e) : \mathit{bool}}\ (\textsc{Pred})$$

$$\frac{\Gamma \vdash e_1 : \mathit{int} \qquad \Gamma \vdash e_2 : \mathit{int}}{\Gamma \vdash -(e_1, e_2) : \mathit{int}}\ (\textsc{Diff}) \qquad \frac{\Gamma \vdash e : T_1 \qquad \Gamma, x : T_1 \vdash \mathit{body} : T_2}{\Gamma \vdash \texttt{let}\ x = e\ \texttt{in}\ \mathit{body} : T_2}\ (\textsc{Var-Intro})$$

$$\frac{\Gamma \vdash e_1 : \mathit{bool} \qquad \Gamma \vdash e_2 : T \qquad \Gamma \vdash e_3 : T}{\Gamma \vdash \texttt{if}\ e_1\ \texttt{then}\ e_2\ \texttt{else}\ e_3 : T}\ (\textsc{If}) \qquad \frac{\Gamma \vdash \mathit{rator} : T_1 \to T_2 \qquad \Gamma \vdash \mathit{rand} : T_1}{\Gamma \vdash (\mathit{rator}\ \mathit{rand}) : T_2}\ (\textsc{Reduc})$$

Each rule reads from the conclusion: to know the type of a difference, check that both operands are integers; the result is an integer. The rule If requires both branches to have the same type, which becomes the type of the whole expression. Reduc (application) requires the operator to be a function whose argument type is the type of the operand.

Functions. The rule for `proc` is Abs:

$$\frac{\Gamma, x : T_1 \vdash \mathit{body} : T_2}{\Gamma \vdash \texttt{proc}\ (x : T_1)\ \mathit{body} : T_1 \to T_2}\ (\textsc{Abs})$$

It needs the type of x. In the typed language the programmer writes it; in Section 5 the checker deduces it. For `letrec`, the function is in scope in its own body, so the context for the body already contains f:

$$\frac{\Gamma, f : T_1 \to T_2, x : T_1 \vdash \mathit{fbody} : T_2 \qquad \Gamma, f : T_1 \to T_2 \vdash \mathit{body} : T}{\Gamma \vdash \texttt{letrec}\ T_2\ f\ (x : T_1) = \mathit{fbody}\ \texttt{in}\ \mathit{body} : T}\ (\textsc{Letrec})$$

A derivation. The rules combine into a tree, the typing derivation. Figure 4 shows the one for `let x = 3 in -(x, 1)`.

$$\dfrac{\dfrac{\dfrac{(x : \mathit{int}) \in \{x : \mathit{int}\}}{x : \mathit{int} \vdash x : \mathit{int}}\ (\textsc{Var}) \qquad \dfrac{}{x : \mathit{int} \vdash 1 : \mathit{int}}\ (\textsc{Const})}{x : \mathit{int} \vdash -(x, 1) : \mathit{int}}\ (\textsc{Diff}) \qquad \dfrac{}{\vdash 3 : \mathit{int}}\ (\textsc{Const})}{\vdash \texttt{let}\ x = 3\ \texttt{in}\ -(x, 1) : \mathit{int}}\ (\textsc{Var-Intro})$$

Figure 4: Typing derivation of `let x = 3 in -(x, 1)`.

2.4 The function type-of

The rules become a function that computes the type of an expression in an environment. The lecture writes it as

```
fun type-of(tenv, exp) {
  if exp==num && const-exp(num) then int
  if exp==var && var-exp(var) then tenv(var)
  if type-of(exp) == int then t(zero?-exp(exp), bool)
  if type-of(exp1) == int && type-of(exp2) == int then t(diff(exp1 exp2), int)
  if type-of(exp) == T1 && type-of(body) == T2 && tenv(var) then t(let-exp(var exp
    body), T2)
  if type-of(exp1)==bool && type-of(exp2)==T && type-of(exp3)==T then t(if-exp(exp1,
    exp2, exp3), T)
  if type-of(rator) == T1 -> T2 && type-of(rand) == T1 then t(call-exp(rator, rand),
    T2)
}
```

In Rascal there is one equation per rule, and each `checkEqual` is a premise:

```
alias TEnv = map[str, Type];

void checkEqual(Type t1, Type t2, Exp e) {
  if (t1 != t2) throw "Types did not match in <e>: <show(t1)> != <show(t2)>";
}

Type typeOf(const(_), TEnv _)   = intType();
Type typeOf(boolean(_), TEnv _) = boolType();
Type typeOf(var(x), TEnv env)   = env[x];
```

```
Type typeOf(e:zero(arg), TEnv env) {
  checkEqual(typeOf(arg, env), intType(), e);
  return boolType();
}

Type typeOf(e:diff(l, r), TEnv env) {
  checkEqual(typeOf(l, env), intType(), e);
  checkEqual(typeOf(r, env), intType(), e);
  return intType();
}

Type typeOf(let(x, v, body), TEnv env)
  = typeOf(body, env + (x: typeOf(v, env)));

Type typeOf(e:ifExp(c, t, f), TEnv env) {
  checkEqual(typeOf(c, env), boolType(), e);
  Type tt = typeOf(t, env);
  checkEqual(typeOf(f, env), tt, e);
  return tt;
}

Type typeOf(proc(x, aType(tx), body), TEnv env)
  = procType(tx, typeOf(body, env + (x: tx)));

Type typeOf(e:call(rator, rand), TEnv env) {
  Type tf = typeOf(rator, env);
  if (procType(targ, tres) := tf) {
    checkEqual(typeOf(rand, env), targ, e);
    return tres;
  }
  throw "Rator not a procedure in <e>: <show(tf)>";
}

Type typeOf(e:letrec(aType(tres), f, x, aType(targ), fBody, body), TEnv env) {
  TEnv env2 = env + (f: procType(targ, tres));
  checkEqual(typeOf(fBody, env2 + (x: targ)), tres, e);
  return typeOf(body, env2);
}

default Type typeOf(Exp e, TEnv _) { throw "Missing type annotation in <e>"; }
```

The tests type the program `double` as `int -> int`, its application to 6 as `int`, and reject an `if` whose branches have different types and the application of a number.

Type consistency. The core of the checker is the comparison of types. Given two types that an expression requires to be equal, the checker continues if they are and reports an error otherwise:

```
error check-equal(T1, T2, exp) {
  if T1 != T2 then
    error("Types did not match in expression %s: %s != %s", exp, name(T1), name(T2))
  else continue
}
```

The Rascal version is `checkEqual` above. The lecture also shows the same checks written with TypePal, a Rascal library for type checkers, where each construct `collect`s its constraints and a solver computes the types.

2.5 Type checking and type inference

Type checking. At compile time all the entities of the program (values, variables, functions, objects) must have a type. The programmer provides the types; the type checker deduces the types of the other expressions and checks that they are consistent. In

```
int res;
int x = 3;
res = x / 2.0;
```

the types of `res` and `x` are given. The checker deduces that `x / 2.0` is `float`, since dividing an `int` by a `float` gives a `float`, and then has to decide whether a `float` can be assigned to an `int` variable. Java rejects it; C converts the value implicitly.

Type inference. The programmer does not provide all the types, and the checker infers them from their use. In Haskell,

```
doubleSmallNumber x = if x > 100
                      then x
                      else x * 2
```

has no type annotation. `x > 100` needs `x` to be something that can be compared, and `x * 2` something that can be multiplied, so the function goes from numbers to numbers. The lecture writes this as $Num \to Num$; GHC's exact answer is `(Ord a, Num a) => a -> a`: for any type `a` that can be compared and has arithmetic. Section 5 shows how such types are computed.

2.6 Type environments

Variables are typed in an environment, the same way the symbol table works for names. The lecture's example is in OCaml:

```
let someval = "hello";;
let otherval = "world" in let someval = 3 in
  Printf.printf "%d %s \n" someval otherval;;
```

The first line gives `someval : string`. The second creates a new environment for `otherval` and, inside it, another one where a new `someval` of type `int` hides the first. The program prints `3 world`. Figure 5 shows the three environments. The same structure in Scheme:

```
(define someval "hello")
(define (inner)
  (let ((otherval "world"))
    (let ((someval 3))
      (list someval otherval))))
```

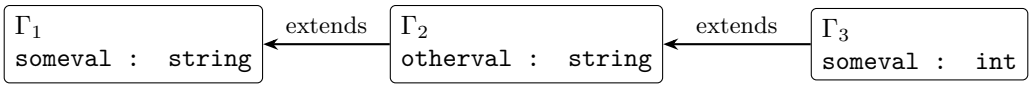


Figure 5: Nested type environments. In Γ_3, $\Gamma_3 \vdash$ `someval` : *int* because the search stops at the innermost declaration.

2.7 Exercises

Exercise 2.1. What would be the rule for the definition of functions in LETREC? How does the checker know the type of the parameter?

Solution. It is ABS: if the body has type T_2 when the parameter has type T_1, the function has type $T_1 \to T_2$. The parameter's type is written by the programmer in the typed language (`proc (x : int) ...`); without the annotation it has to be inferred from how the body uses x.

Exercise 2.2. Give the typing derivation of `letrec int double (x : int) = if zero?(x) then 0 else -((double -(x,1)), -2) in double`.

Solution. With $\Gamma = \{\mathit{double} : \mathit{int} \to \mathit{int}\}$ and $\Gamma' = \Gamma, x : \mathit{int}$. In Γ': `zero?`(x) : *bool* by PRED and VAR; $0 : \mathit{int}$; $-(x, 1) : \mathit{int}$ by DIFF; $(\mathit{double}\ -(x, 1)) : \mathit{int}$ by REDUC, since $\mathit{double} : \mathit{int} \to \mathit{int}$; $-(\ldots, -2) : \mathit{int}$ by DIFF; so the `if` has type *int* by IF, which matches the declared result. Then the body `double` has type $\mathit{int} \to \mathit{int}$ in Γ by VAR, and by LETREC the whole program has type $\mathit{int} \to \mathit{int}$.

Exercise 2.3. Which rule fails for `if zero?(1) then 1 else zero?(0)`?

Solution. IF: the condition is a *bool*, but the branches have types *int* and *bool*, and the rule requires the same type T for both. The Rascal checker reports `Types did not match ...: bool != int`.

Exercise 2.4. In the OCaml example, what does `Printf.printf "%d %s" someval otherval` print if the inner `let someval = 3` is removed? Does the program still type check?

Solution. It does not type check: `someval` is now the outer one, a `string`, and `%d` requires an `int`. The format string makes the types of the arguments part of the check.

3 Lambda calculus

This section and the next go beyond the slides; they give the formal basis for the rule ABS and for Section 9. The lambda calculus is a formal system in which everything is a function. A LETREC `proc (x) body` is a lambda abstraction $\lambda x.\, \mathit{body}$, and the application `(f e)` is the application $f\ e$.

3.1 Syntax

There are three constructs: variables $x, y, z, \ldots$; abstraction $\lambda x.\, e$, a function with parameter x and body e; and application $e_1\, e_2$, which applies e_1 to e_2. Application associates to the left ($f\, a\, b$ is

$(f\,a)\,b$), and the body of an abstraction extends as far to the right as possible. Figure 6 shows the abstract syntax tree of $(\lambda x.\,x\,x)\,(\lambda y.\,y)$.

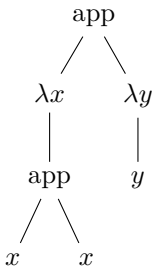


Figure 6: Abstract syntax tree of $(\lambda x.\,x\,x)\,(\lambda y.\,y)$.

Free and bound variables. In $\lambda x.\,M$ the variable x is **bound**: the abstraction is its scope. A variable that is not inside an abstraction for it is **free**. In $\lambda x.\,x\,y$, x is bound and y is free. The name of a bound variable does not matter, as long as renaming does not capture another variable: $\lambda x.\,x \equiv_\alpha \lambda y.\,y$ (α-equivalence).

3.2 Reduction

Computation is one rule, β-reduction, which substitutes the argument into the body:

$$(\lambda x.\,e)\,v \;\to_\beta\; e[x := v].$$

A second rule, η-reduction, says that a function that only passes its argument on is the function itself: $\lambda x.\,(f\,x) \to_\eta f$ when x is not free in f.

For $(\lambda x.\,x\,x)\,(\lambda y.\,y)$:

$$(\lambda x.\,x\,x)\,(\lambda y.\,y) \;\to_\beta\; (\lambda y.\,y)\,(\lambda y.\,y) \;\to_\beta\; \lambda y.\,y.$$

The result has no more redexes (subterms of the form $(\lambda x.\,e)\,v$); it is in normal form.

Evaluation strategies. When a term has several redexes, the order matters. Normal order reduces the leftmost, outermost redex first, and finds a normal form whenever one exists. Call by value evaluates the argument before the call, as C, Java and Python do. Call by name passes the argument unevaluated, which is the basis of lazy evaluation. The runtime block returns to these strategies with the lecture's examples.

3.3 Church encodings

The pure calculus has no numbers or booleans, but they can be encoded as functions. A boolean chooses between two alternatives:

$$\mathit{TRUE} \equiv \lambda t.\,\lambda f.\,t \qquad \mathit{FALSE} \equiv \lambda t.\,\lambda f.\,f \qquad \mathit{AND} \equiv \lambda p.\,\lambda q.\,p\,q\,p.$$

A Church numeral n applies a function n times:

$$0 \equiv \lambda f.\,\lambda x.\,x \qquad 1 \equiv \lambda f.\,\lambda x.\,f\,x \qquad 2 \equiv \lambda f.\,\lambda x.\,f\,(f\,x) \qquad \mathit{SUCC} \equiv \lambda n.\,\lambda f.\,\lambda x.\,f\,(n\,f\,x).$$

3.4 Recursion

Abstractions are anonymous, so a function cannot call itself by name. LETREC solves this with `letrec`, which puts the function in its own scope. The pure calculus uses a fixed-point combinator:

$$Y = \lambda f.\,(\lambda x.\,f\,(x\,x))\,(\lambda x.\,f\,(x\,x)).$$

One step of reduction shows that $Y\,g \to_\beta (\lambda x.\,g\,(x\,x))\,(\lambda x.\,g\,(x\,x)) \to_\beta g\,((\lambda x.\,g\,(x\,x))\,(\lambda x.\,g\,(x\,x)))$, and the argument of g is again the term that $Y\,g$ reduced to. So $Y\,g$ behaves as $g\,(Y\,g)$: it gives g a copy of itself.

3.5 Exercises

Exercise 3.1. Define *OR* and *NOT* with Church booleans and check that *OR TRUE FALSE* reduces to *TRUE*.

Solution. $OR \equiv \lambda p.\,\lambda q.\,p\,p\,q$ and $NOT \equiv \lambda p.\,p\,FALSE\,TRUE$. Then $OR\ TRUE\ FALSE \to_\beta TRUE\ TRUE\ FALSE \to_\beta TRUE$, because *TRUE* returns its first argument.

Exercise 3.2. Show that *SUCC* 1 reduces to 2.

Solution. $SUCC\ 1 \to_\beta \lambda f.\,\lambda x.\,f\,(1\,f\,x)$, and $1\,f\,x = (\lambda f.\,\lambda x.\,f\,x)\,f\,x \to_\beta f\,x$, so the result is $\lambda f.\,\lambda x.\,f\,(f\,x) = 2$.

Exercise 3.3. Which variables are free in $\lambda x.\,(\lambda y.\,x\,y\,z)\,y$?

Solution. z, and the last y, which is outside the abstraction λy. The first y and x are bound.

4 Simply typed lambda calculus

The untyped calculus accepts terms such as *TRUE* 3 or $x\,x$. The **simply typed lambda calculus** (STLC) adds the smallest type discipline: every term has a type, and an application is legal only when the types match. Its rules are the rules of LETREC (Section 2) for variables, functions, application and `if`; LETREC adds numbers, `let` and recursion.

4.1 Types and terms

Types are built from two base types and the function type:

$$\tau ::= \mathbf{int} \mid \mathbf{bool} \mid \tau \to \tau.$$

Terms are those of the untyped calculus, with an annotation on each parameter, plus literals and conditionals:

$$e ::= x \mid \lambda x{:}\tau.\,e \mid e\,e \mid n \mid \mathbf{if}\ e\ \mathbf{then}\ e\ \mathbf{else}\ e.$$

4.2 The typing judgment

$$\frac{x : \tau \in \Gamma}{\Gamma \vdash x : \tau}\ \text{T-Var} \qquad \frac{\Gamma, x : \tau_1 \vdash e : \tau_2}{\Gamma \vdash \lambda x{:}\tau_1.\, e : \tau_1 \to \tau_2}\ \text{T-Abs} \qquad \frac{\Gamma \vdash e_1 : \tau_1 \to \tau_2 \qquad \Gamma \vdash e_2 : \tau_1}{\Gamma \vdash e_1\, e_2 : \tau_2}\ \text{T-App}$$

$$\frac{\Gamma \vdash e_1 : \mathbf{bool} \qquad \Gamma \vdash e_2 : \tau \qquad \Gamma \vdash e_3 : \tau}{\Gamma \vdash \mathbf{if}\ e_1\ \mathbf{then}\ e_2\ \mathbf{else}\ e_3 : \tau}\ \text{T-If}$$

The rules are syntax-directed: for each form of term exactly one rule applies, so checking a term never requires a guess. T-App rejects $3\,(\lambda x{:}\mathbf{int}.\, x)$, because 3 does not have a function type, and $(\lambda x{:}\mathbf{int}.\, x)\ \mathit{true}$, because $\mathbf{int} \neq \mathbf{bool}$. Figure 7 shows a derivation.

$$\dfrac{\dfrac{\dfrac{x : \mathbf{int} \in \{x : \mathbf{int}, y : \mathbf{bool}\}}{x : \mathbf{int}, y : \mathbf{bool} \vdash x : \mathbf{int}}\ \text{T-Var}}{x : \mathbf{int} \vdash \lambda y{:}\mathbf{bool}.\, x : \mathbf{bool} \to \mathbf{int}}\ \text{T-Abs}}{\vdash \lambda x{:}\mathbf{int}.\, \lambda y{:}\mathbf{bool}.\, x : \mathbf{int} \to \mathbf{bool} \to \mathbf{int}}\ \text{T-Abs}$$

Figure 7: Typing derivation of $\lambda x{:}\mathbf{int}.\, \lambda y{:}\mathbf{bool}.\, x$.

4.3 Capture-avoiding substitution

β-reduction substitutes the argument into the body, $e[x := v]$, and the substitution must not capture variables. With $e = \lambda y.\, x\, y$ and $v = y$, replacing x by y textually gives $\lambda y.\, y\, y$, where the y that was free has been captured by λy. The fix is to rename the bound variable first ($\lambda z.\, x\, z$, which is α-equivalent) and then substitute: $\lambda z.\, y\, z$. Compilers often avoid the problem by replacing names with de Bruijn indices, the distance to the binder.

4.4 Strong normalization

Theorem 4.1 (Strong normalization)**.** *If $\vdash e : \tau$, every reduction sequence that starts at e ends in a normal form.*

So every well-typed STLC program terminates, and STLC is not Turing complete. The reason is that self-application has no type. For $x\, x$ to be typed, x needs a type α with $\alpha = \alpha \to \beta$, and no finite type satisfies that equation, because the right side is always bigger than the left. The combinator Y is built from self-applications, so it has no type either. Section 5 meets the same equation as the occurs check of `proc (f) zero?((f f))`, and Section 8 adds recursive types to give it a solution. LETREC recovers recursion directly with `letrec`, and that is why it can loop.

4.5 Propositions as types

Read as logical formulas, the types of STLC are propositions of intuitionistic logic, and the well-typed terms are their proofs (Table 3). T-App is modus ponens: from a proof of $P \Rightarrow Q$ and a proof of P it gives a proof of Q. T-Abs is conditional proof: to prove $P \Rightarrow Q$, assume P and derive Q.

The term $\lambda x{:}\tau_1.\, \lambda y{:}\tau_2.\, x$ of Figure 7 is a proof of $P \Rightarrow (Q \Rightarrow P)$. Proof assistants such as Coq and Agda are built on this correspondence: checking the type of a program is checking a proof.

Logic	Types
proposition	type
proof of P	term of type P
implication $P \Rightarrow Q$	function type $P \to Q$
conjunction $P \wedge Q$	product type $P \times Q$
modus ponens	application

Table 3: The Curry–Howard correspondence.

4.6 Exercises

Exercise 4.2. Is $\lambda x{:}(\mathbf{int} \to \mathbf{int}).\, x\, x$ well typed?

Solution. No. T-App on $x\, x$ needs the type of the operator, $\mathbf{int} \to \mathbf{int}$, to have as argument type the type of the operand, $\mathbf{int} \to \mathbf{int}$. But the argument type is $\mathbf{int}$, and $\mathbf{int} \neq \mathbf{int} \to \mathbf{int}$.

Exercise 4.3. Perform $(\lambda y.\, x\, (\lambda x.\, x\, y))[x := y\, z]$.

Solution. The outer λy would capture the y of $y\, z$, so rename it to w: $\lambda w.\, x\, (\lambda x.\, x\, w)$. Inside λx the variable x is bound, so only the first x is replaced: $\lambda w.\, (y\, z)\, (\lambda x.\, x\, w)$.

Exercise 4.4. Write a term of type $(\tau_1 \to \tau_2) \to (\tau_2 \to \tau_3) \to (\tau_1 \to \tau_3)$. Which logical rule does it prove?

Solution. $\lambda f{:}\tau_1 \to \tau_2.\, \lambda g{:}\tau_2 \to \tau_3.\, \lambda x{:}\tau_1.\, g\, (f\, x)$, function composition. It proves that implication is transitive: if $P \Rightarrow Q$ and $Q \Rightarrow R$, then $P \Rightarrow R$.

5 Type inference

Writing the types in a program helps with design and documentation, but it takes time. In a statically typed language the compiler can deduce the types from the context in which each entity is used. In LETREC this means making the annotations optional:

```
Expression ::= letrec Type? Identifier (Identifier : Type?) = Expression in
    Expression
             | proc (Identifier : Type?) Expression
             | ...
Type       ::= int | boolean
```

The Rascal grammar of Section 2 already allows a missing type; the abstract syntax stores it as `noType()`.

5.1 Rules with type variables

When a type is unknown, the checker gives it a name, a **type variable** $t_0, t_1, \ldots$, and collects equations between types from the rules:

$$\frac{e_1 : \mathit{Int} \qquad e_2 : \mathit{Int}}{-(e_1, e_2) : \mathit{Int}}\ (\textsc{Diff}) \qquad\qquad \frac{e_1 : \mathit{Int}}{\texttt{zero?}(e_1) : \mathit{Boolean}}\ (\textsc{Pred})$$

$$\frac{e_1 : \mathit{Boolean} \qquad e_2 : T \qquad e_3 : T}{(\texttt{if}\ e_1\ \texttt{then}\ e_2\ \texttt{else}\ e_3) : T}\ (\textsc{If}) \qquad\qquad \frac{v : T_{var} \qquad \mathit{body} : T_{body}}{\texttt{proc}(v)\ \mathit{body} : T_{var} \to T_{body}}\ (\textsc{Abs})$$

$$\frac{\mathit{rator} : T_1 \to T_2 \qquad \mathit{rand} : T_1}{\texttt{call}(\mathit{rator}, \mathit{rand}) : T_2}\ (\textsc{Reduc})$$

Every use of a rule produces equations. Solving the equations gives the types.

Example 1. The lecture starts with

```
letrec foo (x) = if zero?(x) then 1 else -(x, (foo -(x,1))) in foo
```

Let $x : t_x$ and $\mathit{foo} : t_x \to t_r$. `zero?`$(x)$ gives $t_x = \mathit{Int}$. The `then` branch is $1 : \mathit{Int}$, so the `if` has type Int and $t_r = \mathit{Int}$. The `else` branch is a difference, which needs $(\mathit{foo}\ -(x, 1)) : \mathit{Int}$, consistent with $t_r = \mathit{Int}$. So $\mathit{foo} : \mathit{Int} \to \mathit{Int}$.

Example 2. For

```
proc (f) proc (x) -((f 3), (f x))
```

the lecture names every subexpression (Figure 8): t_0 for the whole term, t_1 for the inner `proc`, t_2 for the difference, t_3 for `(f 3)`, t_4 for `(f x)`, and t_f, t_x for the parameters. The equations are in Table 4.

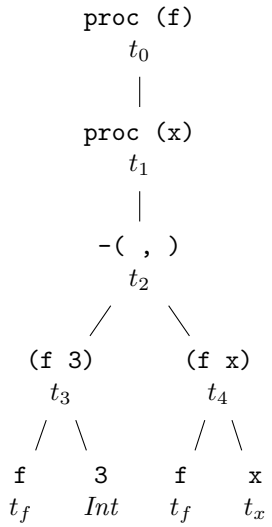


Figure 8: Type variables for the subexpressions of `proc (f) proc (x) -((f 3), (f x))`.

Subexpression	Rule	Equation
`(f 3)`	Reduc	$t_f = Int \to t_3$
`(f x)`	Reduc	$t_f = t_x \to t_4$
`-((f 3),(f x))`	Diff	$t_3 = Int,\ t_4 = Int,\ t_2 = Int$
`proc (x) ...`	Abs	$t_1 = t_x \to t_2$
`proc (f) ...`	Abs	$t_0 = t_f \to t_1$

Table 4: Equations for Example 2.

Solving: $t_f = Int \to Int$; from the second equation $t_x \to Int = Int \to Int$, so $t_x = Int$; then $t_1 = Int \to Int$ and

$$t_0 = (Int \to Int) \to Int \to Int.$$

The function takes a function from integers to integers and returns a function from integers to integers. The parentheses around the first $Int \to Int$ matter: the slide writes $t_0 = Int \to Int \to Int \to Int$, which, since the arrow associates to the right, would be a function of three integers.

Example 3. `if x then -(x,1) else 0`. The difference needs $t_x = Int$; the condition of the `if` needs $t_x = Boolean$. Together, $Boolean = Int$, which has no solution: the expression has a type error.

Example 4. `proc (f) zero?((f f))`. With $f : t_f$ and $(f\ f) : t_3$, application gives $t_f = t_f \to t_3$, and `zero?` gives $t_3 = Int$, so $t_f = t_f \to Int$. The type of f would be defined in terms of itself, and no finite type satisfies the equation. The **occurs check** detects this: a variable cannot be bound to a type that contains it.

5.2 The algorithm

The lecture divides type inference into five parts:

1. substitution: replace a type variable by a known type inside a type expression;
2. the no-occurrence property: a variable that is bound never appears in the types of the substitution, which the occurs check preserves;
3. unification: add the equation $t_1 = t_2$ between two types to a substitution;
4. fresh type variables for the expressions whose type is unknown;
5. the type of an expression, computed from the rules and the current substitution.

Substitution. Substituting a type t_1 for a variable t_{var} in a type expression t_0 replaces every occurrence:

$$(t_{var} \to t_x \to t_y \to Int \to t_{var})[t_{var} = T] \ = \ T \to t_x \to t_y \to Int \to T.$$

The function goes by cases on t_0:

```
substitution(t0, tvar, t1) : Type {
  switch (t0) {
    case int-type(): int-type;
    case boolean-type(): boolean-type;
    case proc-type(arg-type, result-type):
      proc-type(substitution(arg-type, tvar, t1), substitution(result-type, tvar, t1)
    );
    case tvar-type(sn): if (t0 == tvar) then t1 else t0;
  }
}
```

(The slide compares `t1 = tvar` in the last case. The comparison is between the variable found, t_0, and the variable being replaced.) A substitution maps several variables to types. In Rascal, `apply` follows chains such as $t_1 = t_2$, $t_2 = Int$:

```
alias Subst = map[int, Type];

Type substitution(intType(), int _, Type _)  = intType();
Type substitution(boolType(), int _, Type _) = boolType();
Type substitution(procType(a, r), int n, Type t1)
  = procType(substitution(a, n, t1), substitution(r, n, t1));
Type substitution(tvar(m), int n, Type t1)   = m == n ? t1 : tvar(m);

Type apply(tvar(n), Subst s)       = n in s ? apply(s[n], s) : tvar(n);
Type apply(procType(a, r), Subst s) = procType(apply(a, s), apply(r, s));
default Type apply(Type t, Subst _) = t;
```

Unification. Given two types, the unifier returns a substitution in which they are equal:

1. apply the current substitution to t_1 and t_2;
2. if the results are the same, return the substitution unchanged;
3. if t_1 is a variable, propose $t_1 = t_2$, after the occurs check;
4. if t_2 is a variable, do the same;
5. if both are function types, unify the argument types and then the result types; otherwise (*Int* against *Boolean*, or a basic type against a function type) there is no solution.

```
unifier(t1, t2, subs, exp) : Substitution {
  t1 = apply-substitution(t1, subs);
  t2 = apply-substitution(t2, subs);
  condition (t1 == t2) { subs; }
  tvar-type?(t1) {
    if (no-occurrence?(t1, t2)) extend-substitution(subs, t1, t2);
    else Error(No occurrence violation t1 t2 exp);
  }
  tvar-type?(t2) { ... }
  proc-type?(t1) && proc-type?(t2) {
    subs = unifier(proc-type.arg(t1), proc-type.arg(t2), subs, exp);
    subs = unifier(proc-type.res(t1), proc-type.res(t2), subs, exp);
```

```
  }
  else { Error(Unification failure t1 t2 exp); }
}

no-occurrence?(tvar, t) : Boolean {
  switch (t) {
    int-type: true;
    bool-type: true;
    proc-type(args, res): no-occurrence?(tvar, args) && no-occurrence?(tvar, res);
    var-type: not(tvar == t)
  }
}
```

In Rascal:

```
bool noOccurrence(int n, tvar(m))         = n != m;
bool noOccurrence(int n, procType(a, r))  = noOccurrence(n, a) && noOccurrence(n, r);
default bool noOccurrence(int _, Type _)  = true;

Subst unifier(Type t1, Type t2, Subst s, Exp e) {
  t1 = apply(t1, s);
  t2 = apply(t2, s);
  if (t1 == t2) return s;
  if (tvar(n) := t1) {
    if (noOccurrence(n, t2)) return s + (n: t2);
    throw "No-occurrence violation: <show(t1)> occurs in <show(t2)>";
  }
  if (tvar(n) := t2) {
    if (noOccurrence(n, t1)) return s + (n: t1);
    throw "No-occurrence violation: <show(t2)> occurs in <show(t1)>";
  }
  if (procType(a1, r1) := t1, procType(a2, r2) := t2) {
    s = unifier(a1, a2, s, e);
    return unifier(r1, r2, s, e);
  }
  throw "Unification failure: <show(t1)> and <show(t2)>";
}
```

Types for expressions. A missing annotation becomes a fresh variable (`optionaltype-2-type`), and the inference function follows the rules, calling the unifier at each premise that requires two types to be equal. For an application it creates a variable for the result and unifies the type of the operator with $t_{rand} \rightarrow t_{res}$:

```
int counter = 0;
Type fresh() { counter += 1; return tvar(counter); }

Type otype2type(noType())   = fresh();
Type otype2type(aType(t))   = t;

alias Result = tuple[Type t, Subst s];

Result infer(const(_), TEnv _, Subst s)   = <intType(), s>;
Result infer(boolean(_), TEnv _, Subst s) = <boolType(), s>;
```

```
Result infer(var(x), TEnv env, Subst s)   = <env[x], s>;

Result infer(e:diff(l, r), TEnv env, Subst s) {
  <tl, s> = infer(l, env, s);  s = unifier(tl, intType(), s, e);
  <tr, s> = infer(r, env, s);  s = unifier(tr, intType(), s, e);
  return <intType(), s>;
}

Result infer(e:zero(arg), TEnv env, Subst s) {
  <ta, s> = infer(arg, env, s);  s = unifier(ta, intType(), s, e);
  return <boolType(), s>;
}

Result infer(e:ifExp(c, t, f), TEnv env, Subst s) {
  <tc, s> = infer(c, env, s);  s = unifier(tc, boolType(), s, e);
  <tt, s> = infer(t, env, s);
  <tf, s> = infer(f, env, s);  s = unifier(tt, tf, s, e);
  return <tt, s>;
}

Result infer(let(x, v, body), TEnv env, Subst s) {
  <tv, s> = infer(v, env, s);
  return infer(body, env + (x: tv), s);
}

Result infer(proc(x, ot, body), TEnv env, Subst s) {
  Type tx = otype2type(ot);
  <tb, s> = infer(body, env + (x: tx), s);
  return <procType(tx, tb), s>;
}

Result infer(e:call(rator, rand), TEnv env, Subst s) {
  Type tres = fresh();
  <tf, s> = infer(rator, env, s);
  <ta, s> = infer(rand, env, s);
  s = unifier(tf, procType(ta, tres), s, e);
  return <tres, s>;
}

Result infer(e:letrec(ores, f, x, ox, fBody, body), TEnv env, Subst s) {
  Type tres = otype2type(ores);
  Type tx = otype2type(ox);
  TEnv env2 = env + (f: procType(tx, tres));
  <tb, s> = infer(fBody, env2 + (x: tx), s);
  s = unifier(tb, tres, s, e);
  return infer(body, env2, s);
}
```

The tests run the four examples of this section: `foo` gets `int -> int`, Example 2 gets `(int -> int) -> int -> int`, Example 3 fails with a unification failure and Example 4 with a no-occurrence violation.

5.3 Principal types and Algorithm W

This subsection goes beyond the slides. The method above is a version of Algorithm W, the inference algorithm of ML and Haskell (the Hindley–Milner type system). Unification, when it succeeds, finds the **most general unifier**: every other solution is an instance of it. So every typable term has a **principal type**, of which all its other types are instances. The principal type of `proc (x) x` is $t_0 \to t_0$; $Int \to Int$ is a valid type for it, but an instance. The Rascal test `identity` shows the variable left in the result. Section 6 uses this to give `let`-bound functions polymorphic types.

Checking and inference are different problems. Checking verifies a type that is given; inference has to find one. For richer type systems inference can be undecidable, and many modern languages (Rust, Swift, TypeScript) use **bidirectional** checking: some constructs synthesize their type bottom-up (variables, literals, applications), and others are checked against a type that comes from the context (an unannotated function passed where a function type is expected). They need more annotations than ML, and combine better with subtyping.

Generics in Java. Java infers the type parameters at each call of a generic method, a restricted form of the same problem:

```
record MyPair<A, B>(A x, B y) {}

static <T> T id(T x) { return x; }

static <A> A head(List<A> lst) { return lst.get(0); }
```

```
var p = new MyPair<>(1, "Java");
var q = id(1);
var result = head(List.of(1, 2, 3));
var pairs = List.of(new MyPair<>(1, "a"), new MyPair<>(2, "b"));
```

For `p` the compiler infers `MyPair<Integer, String>`, for `q` and `result` `Integer`, and for `pairs` `List<MyPair<Integer, String>>`.

References. The same method extends to the commands of the semantics block. A cell has type **ref** T, and commands have type **unit**:

$$\frac{\Gamma \vdash l : \mathbf{ref}\ T}{\Gamma \vdash\, !l : T}\ \text{T-Deref} \qquad \frac{\Gamma \vdash l : \mathbf{ref}\ T \qquad \Gamma \vdash e : T}{\Gamma \vdash l := e : \mathbf{unit}}\ \text{T-Assign}$$

$$\frac{\Gamma \vdash B : \mathbf{bool} \qquad \Gamma \vdash C : \mathbf{unit}}{\Gamma \vdash \mathbf{while}\ B\ \mathbf{do}\ C : \mathbf{unit}}\ \text{T-While}$$

In the program

$a := 1\ ;\ b := 1\ ;\ \mathbf{while}\ !n < 5\ \mathbf{do}\ C$
$C = \mathbf{if}\ !term \le 2\ \mathbf{then}\ res :=!n\ \mathbf{else}\ (c :=!a+!b\ ;\ res :=!n\ ;\ a :=!b\ ;\ b :=!c)\ ;\ n :=!n + 1$

every cell is dereferenced and compared with or added to an integer, so $n, res, a, b, c, term$: **ref int**; the conditions are **bool**; and every command, the `while` included, is **unit**.

5.4 Exercises

Exercise 5.1. What is the inferred type of `if x then -(x,1) else 0`?

Solution. None. The difference requires $t_x = \mathit{Int}$ and the condition requires $t_x = \mathit{Boolean}$; unifying *Int* with *Boolean* fails.

Exercise 5.2. Infer the type of `proc (f) proc (x) (f (f x))`.

Solution. $(f\ x)$ gives $t_f = t_x \to t_1$ and $(f\ (f\ x))$ gives $t_f = t_1 \to t_2$. Unifying the two function types: $t_x = t_1$ and $t_1 = t_2$. So $t_f = t_x \to t_x$ and the type is $(t_x \to t_x) \to t_x \to t_x$: apply a function twice, for any type.

Exercise 5.3. Infer the principal type of $\lambda x.\, \lambda y.\, \lambda z.\, x\, z\, (y\, z)$ (the S combinator).

Solution. Let $z : \gamma$. $y\, z$ gives $y : \gamma \to \beta$. $x\, z\, (y\, z)$ gives $x : \gamma \to \beta \to \alpha$. The type is $(\gamma \to \beta \to \alpha) \to (\gamma \to \beta) \to \gamma \to \alpha$.

Exercise 5.4. Run the unifier by hand on $t_f = t_f \to \mathit{Int}$ and say at which step it stops.

Solution. After applying the (empty) substitution the two types are different, and t_f is a variable, so the unifier proposes $t_f = t_f \to \mathit{Int}$. The occurs check finds t_f inside $t_f \to \mathit{Int}$ and reports a no-occurrence violation. Without the check, the substitution would replace t_f forever.

6 Polymorphism

This section goes beyond the slides. In STLC and in typed LETREC, the identity function has to be written once per type: `proc (x : int) x` and `proc (x : bool) x` are different programs. Section 5 showed that inference gives `proc (x) x` the type $t_0 \to t_0$, with a variable left over. **Polymorphism** is the name for the features that let one piece of code work at many types. There are three classical kinds:

- parametric: one implementation that behaves the same at every type;
- ad hoc: one name with a different implementation per type (overloading, type classes);
- subtype: one implementation usable at every subtype of the type it expects (Section 7).

6.1 Parametric polymorphism

System F abstracts over types the same way λ abstracts over values. It adds the type $\forall\alpha.\, \tau$, type abstraction $\Lambda\alpha.\, e$ and type application $e\,[\tau]$:

$$\mathit{id} = \Lambda\alpha.\, \lambda x{:}\alpha.\, x \ :\ \forall\alpha.\, \alpha \to \alpha.$$

$\mathit{id}\,[\mathbf{int}]$ 3 reduces to 3 and $\mathit{id}\,[\mathbf{bool}]$ *true* to *true*: one term, instantiated twice (Figure 9). Hindley–Milner, the system behind ML and Haskell, is the fragment of System F where $\forall$ only appears at the outside of a type and instantiation is implicit. That restriction is what makes the inference of Section 5 possible.

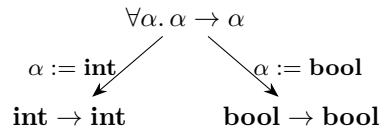


Figure 9: A polymorphic type and two of its instances.

Generalization happens at `let`. In Haskell

```
pair :: (Int, Bool)
pair = let ident = \x -> x
       in (ident 3, ident True)
```

type checks: the `let`-bound `ident` gets the type $\forall\alpha.\,\alpha \to \alpha$, and each use instantiates it. The same thing with a parameter, `(\i -> (i 3, i True)) (\x -> x)`, is rejected with `No instance for 'Num Bool'`: a parameter has one type in its whole body, so after `i 3` fixes it to a function on numbers, `i True` fails. Algorithm W quantifies the free type variables of a `let`-bound value (they cannot appear in the context), and never those of a parameter. ML restricts generalization further to values (the value restriction): a generalized mutable cell could store a value at one type and be read at another.

6.2 Ad hoc polymorphism

Overloading binds one name to unrelated implementations, chosen at compile time by the types of the arguments:

```
    static int add(int a, int b) { return a + b; }
    static String add(String a, String b) { return a + b; }
```

Haskell's type classes give this a structure: a class declares operations and each instance implements them for one type. A function constrained by a class is parametric in the type and ad hoc in the operation it calls:

```
class MyEq a where
  eq :: a -> a -> Bool

instance MyEq Int where
  eq x y = x == y

instance MyEq Bool where
  eq True  True  = True
  eq False False = True
  eq _     _     = False

allEqual :: MyEq a => [a] -> Bool
allEqual xs = and (zipWith eq xs (drop 1 xs))
```

Java gets the same effect with an interface as a bound on a type parameter (Rust uses traits for this):

```
    interface Speak { String speak(); }
```

```
record Dog() implements Speak { public String speak() { return "Woof"; } }
record Cat() implements Speak { public String speak() { return "Meow"; } }

static <T extends Speak> String greet(T x) {
    return "Hello, " + x.speak();
}
```

6.3 Higher-rank polymorphism

Hindley–Milner only allows $\forall$ at the outside of a type. A function that receives a polymorphic function and uses it at two types needs a $\forall$ inside the argument, a rank-2 type:

```
apply2 :: (forall a. a -> a) -> (Int, Bool)
apply2 f = (f 3, f True)
```

Inference cannot find this type (the first use would fix the type of `f`), but checking against the written signature works. Haskell allows it with the `RankNTypes` extension.

6.4 Exercises

Exercise 6.1. Write the System F term and type of *const*, with *const* $x\ y = x$. What is the normal form of *const* [**int**] [**bool**] 3 *true*?

Solution. $\Lambda\alpha.\,\Lambda\beta.\,\lambda x{:}\alpha.\,\lambda y{:}\beta.\,x : \forall\alpha.\,\forall\beta.\,\alpha \to \beta \to \alpha$. The application reduces to 3.

Exercise 6.2. What does the inference of Section 5 say about `let i = proc (x) x in if (i zero?(0)) then (i 1) else 2`? Would the program type check with let-polymorphism?

Solution. It fails. The inference of Section 5 does not generalize, so `i` has one type $t \to t$ in the whole body. `(i zero?(0))` gives $t = \mathit{Boolean}$ and `(i 1)` gives $t = \mathit{Int}$, and unification fails. With let-polymorphism `i` would get $\forall t.\,t \to t$, each use would have its own instance, and the program would have type *Int*.

7 Subtyping

This section goes beyond the slides, except for the example of variance, which comes from the statement of the course project. **Subtyping** asks when a term of one type can be used where another type is expected. The answer is a relation $\tau_1 <: \tau_2$, read "τ_1 is a subtype of τ_2": anywhere a τ_2 is expected, a τ_1 will do.

7.1 Subsumption

Subtyping enters the type system through one rule:

$$\frac{\Gamma \vdash e : \tau_1 \qquad \tau_1 <: \tau_2}{\Gamma \vdash e : \tau_2}\ \text{T-Sub}$$

The other rules do not change; the design work is in the definition of $<:$.

Records. A record with more fields is a subtype of a record with fewer, if the shared fields agree (width subtyping): $\{name : string, age : \textbf{int}, email : string\} <: \{name : string, age : \textbf{int}\}$. A function that only reads `name` and `age` can ignore `email`. A record is also a subtype if each field is a subtype of the corresponding field (depth subtyping).

7.2 Variance

How does subtyping extend to type constructors? For functions the rule is

$$\frac{\sigma_1 <: \tau_1 \qquad \tau_2 <: \sigma_2}{\tau_1 \to \tau_2 \;<:\; \sigma_1 \to \sigma_2}\ \text{S-Arrow}$$

The argument goes in the opposite direction. A caller that expects $\sigma_1 \to \sigma_2$ will pass any σ_1, so the function must accept at least that: $\sigma_1 <: \tau_1$. The caller will use the result as a σ_2, so the function may return something more specific: $\tau_2 <: \sigma_2$. Function types are **contravariant** in the argument and **covariant** in the result.

The statement of the course project (Objectilang) has an example. With `Puppy <: Dog <: Animal` and `DogFood <: Food`, a generic `Transformer[-In, +Out]` declares `In` contravariant and `Out` covariant, so

$$\texttt{Transformer}[\texttt{Animal}, \texttt{DogFood}] \;<:\; \texttt{Transformer}[\texttt{Puppy}, \texttt{Food}].$$

A transformer that accepts any animal and returns dog food can be used where the program needs one that accepts puppies and returns some food (Figure 10). The comments of the statement say the same thing: the input is broader and the output narrower.

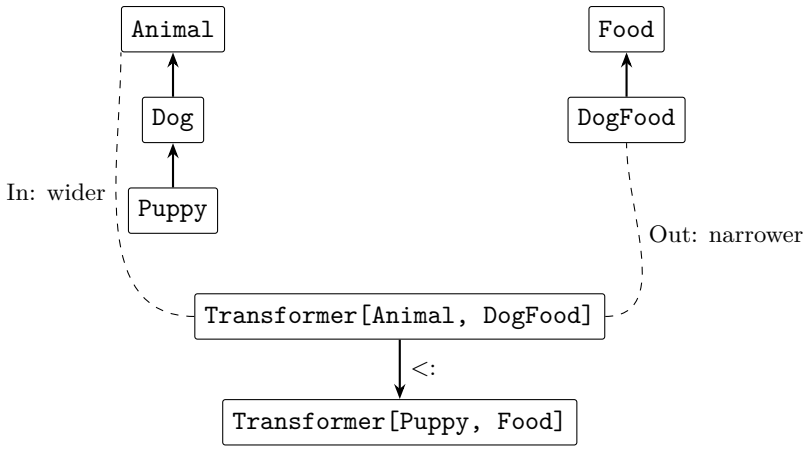


Figure 10: Subtype hierarchies (arrows point to the supertype) and the variance of `Transformer[-In, +Out]`.

Java expresses variance at the use of a generic type, with wildcards:

```
static class Animal {}
static class Dog extends Animal {}

static Animal pick(java.util.List<? extends Animal> animals) { return animals.get
(0); }
```

`List<? extends Animal>` accepts a `List<Dog>`: reading animals from it is safe. Writing is not, which is why a mutable list cannot be covariant without the wildcard.

Liskov substitution. In object-oriented design the same idea is the Liskov substitution principle: if S is a subtype of T, objects of type T can be replaced by objects of type S without breaking the program. Subsumption is the typing side of it; the variance rules keep it sound.

7.3 Nominal and structural subtyping

In a **nominal** system the relation has to be declared. In Java a class is a subtype of an interface only if it says `implements`:

```
interface Flying { String fly(); }

static class Duck implements Flying {
    public String fly() { return "flap flap"; }
}
```

Without `implements Flying`, `Flying f = new Duck();` is rejected with `incompatible types: Duck cannot be converted to Flying`, even though `Duck` has the method. In a **structural** system the shape is enough: in TypeScript and in Go, a value with a method `fly` satisfies an interface that requires it, without mentioning the interface. Duck typing (Section 1) is the dynamic version of structural typing.

7.4 Exercises

Exercise 7.1. Is $\mathit{Animal} \to \mathit{Dog}$ a subtype of $\mathit{Dog} \to \mathit{Animal}$? And the other way around?

Solution. By S-Arrow, $\mathit{Animal} \to \mathit{Dog} <: \mathit{Dog} \to \mathit{Animal}$ needs $\mathit{Dog} <: \mathit{Animal}$ (argument) and $\mathit{Dog} <: \mathit{Animal}$ (result). Both hold, so yes. The other way around needs $\mathit{Animal} <: \mathit{Dog}$, which does not hold.

Exercise 7.2. With `Transformer[-In, +Out]`, which of these relations hold?

1. `Transformer[Dog, Food]` <: `Transformer[Animal, Food]`
2. `Transformer[Animal, DogFood]` <: `Transformer[Dog, DogFood]`

Solution. The first does not hold: `In` is contravariant, so the input on the right (`Animal`) would have to be a subtype of the input on the left (`Dog`). A caller that expects to pass any animal could pass a cat. The second holds: `Dog <: Animal` for the input, and the output is the same.

8 Recursive types

The inductive types of Section 1 are defined by constants and constructors that refer to the type being defined: $0 : \mathbb{N}$ and $s : \mathbb{N} \to \mathbb{N}$. This section, which goes beyond the slides, shows how a type system expresses that self-reference. A list of integers is either empty or an integer followed by another list:

$$\alpha = \mathbf{unit} + (\mathbf{int} \times \alpha).$$

With the types of STLC this equation has no solution. Expanding it never ends, $\alpha = \mathbf{unit} + \mathbf{int} \times (\mathbf{unit} + \mathbf{int} \times (\dots))$, for the same reason that $t_f = t_f \to \mathit{Int}$ failed the occurs check in Section 5.

8.1 μ-types

A new type former names the solution: $\mu\alpha.\,\tau$ is the type α that satisfies $\alpha = \tau$, where τ may mention α.

$$\textbf{List int} = \mu\alpha.\,\textbf{unit} + (\textbf{int} \times \alpha) \qquad \textbf{Tree} = \mu\alpha.\,\textbf{int} + (\alpha \times \textbf{int} \times \alpha) \qquad \mathbb{N} = \mu\alpha.\,\textbf{unit} + \alpha$$

In the last one, **unit** plays the role of 0 and the right side of the sum the role of s.

Fold and unfold. The values of $\mu\alpha.\,\tau$ are related to its unfolding $\tau[\mu\alpha.\tau/\alpha]$ (the body with the whole type put back for α) by two operations:

$$\frac{\Gamma \vdash e : \tau[\mu\alpha.\tau/\alpha]}{\Gamma \vdash \textbf{fold}(e) : \mu\alpha.\,\tau}\ \text{T-Fold} \qquad\qquad \frac{\Gamma \vdash e : \mu\alpha.\,\tau}{\Gamma \vdash \textbf{unfold}(e) : \tau[\mu\alpha.\tau/\alpha]}\ \text{T-Unfold}$$

fold packages one layer (what a constructor such as *cons* does) and **unfold** opens one layer (what a pattern match does). Figure 11 shows the list $[1, 2]$ built with three folds, one per constructor.

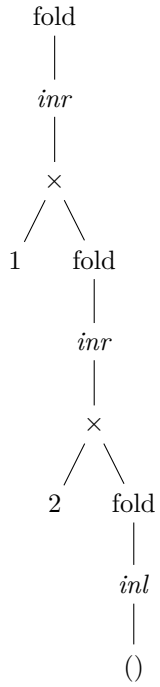


Figure 11: The list $[1, 2]$ as a value of $\mu\alpha.\,\textbf{unit} + (\textbf{int} \times \alpha)$. *inl* and *inr* choose a side of the sum.

Languages with explicit fold and unfold, and a type equality that only compares syntax, are *iso-recursive*; languages where $\mu\alpha.\,\tau$ and its unfolding are the same type are *equi-recursive*, and need a more complex equality on infinite trees. ML-family languages are iso-recursive, which fits their inference.

8.2 Algebraic data types

The data types of Haskell are recursive types where the constructors are the folds and pattern matching is the unfold:

```
data List a = Nil | Cons a (List a)

data Tree a = Leaf a | Node (Tree a) a (Tree a)
```

```
len :: List a -> Int
len Nil         = 0
len (Cons _ xs) = 1 + len xs

size :: Tree a -> Int
size (Leaf _)     = 1
size (Node l _ r) = size l + 1 + size r
```

`Cons` takes an element and a list (one unfolding of the type) and returns a list; the equation `len (Cons _ xs)` opens one layer. Recursive types do not reopen what the occurs check closed: ordinary type variables still cannot refer to themselves, and self-reference is allowed only where a μ (a `data` declaration) introduces it.

8.3 Exercises

Exercise 8.1. Unfold **List int** one level.

Solution. $\mathbf{unit} + (\mathbf{int} \times \mu\alpha.\, \mathbf{unit} + (\mathbf{int} \times \alpha))$, that is, $\mathbf{unit} + (\mathbf{int} \times \mathbf{List\ int})$: empty, or an integer and a list.

Exercise 8.2. Give a μ-type for nonempty lists of booleans.

Solution. $\mu\alpha.\, \mathbf{bool} + (\mathbf{bool} \times \alpha)$: the base case is a single boolean instead of **unit**.

Exercise 8.3. Why does Rust reject `enum List<T> { Nil, Cons(T, List<T>) }` and accept `Cons(T, Box<List<T>>)`?

Solution. Rust stores the fields of an `enum` inline, so a `List` would contain a `List` and have infinite size. `Box` stores a pointer, of fixed size, to a list on the heap. Haskell and ML represent recursive values with pointers by default, which is why they need no annotation.

9 Type safety

This section goes beyond the slides. Type safety is the formal version of "well-typed programs cannot go wrong": a well-typed program never reaches a state where no rule applies. It is proved as two theorems.

Theorem 9.1 (Progress)**.** *If $\vdash e : \tau$, then e is a value or there is an e' with $e \to e'$.*

Theorem 9.2 (Preservation)**.** *If $\vdash e : \tau$ and $e \to e'$, then $\vdash e' : \tau$.*

Together they give safety: start from a well-typed term; progress says it is a value or it can take a step, and preservation says that after the step it is still well typed with the same type, so progress applies again (Figure 12). The evaluation either ends in a value of type τ or runs forever, as in the `letrec` loop of Section 1; it never gets stuck.

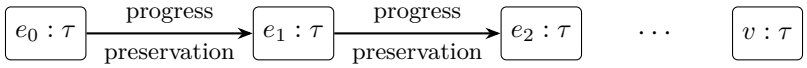


Figure 12: Each step exists by progress and keeps the type by preservation.

9.1 Lemmas

Lemma 9.3 (Canonical forms)**.** *If v is a value and $\vdash v : \tau$, the form of v is determined by τ: if $\tau = \mathbf{bool}$ then v is true or false; if $\tau = \tau_1 \to \tau_2$ then $v = \lambda x{:}\tau_1.\, e$.*

Lemma 9.4 (Substitution)**.** *If $\Gamma, x : \tau' \vdash e : \tau$ and $\Gamma \vdash v : \tau'$, then $\Gamma \vdash e[x := v] : \tau$.*

Canonical forms lets a proof split cases on the form of a value; substitution handles the β-reduction case of preservation: the body e was typed assuming $x : \tau'$, and the argument has type τ'.

9.2 Proof strategy

Both theorems are proved by induction on the typing derivation, with one case per rule. For progress with T-App, $e = e_1\, e_2$: by the induction hypothesis e_1 is a value or takes a step. If it takes a step, so does $e_1\, e_2$. If it is a value, its type is a function type, so by canonical forms it is a λ; then the same reasoning applies to e_2, and when both are values $e_1\, e_2$ is a β-redex and takes a step. The rules of LETREC (Section 2) are the cases a proof for LETREC would cover; the case Diff, for example, uses canonical forms to know that both operands of a difference are numbers when they are values.

9.3 Exercises

Exercise 9.5. Prove progress for T-If, assuming it for the condition.

Solution. Let $e = \mathbf{if}\ e_1\ \mathbf{then}\ e_2\ \mathbf{else}\ e_3$ with $\vdash e_1 : \mathbf{bool}$. By the induction hypothesis e_1 is a value or takes a step. If it takes a step, e steps by reducing the condition. If it is a value, by canonical forms it is *true* or *false*, and e steps to e_2 or e_3.

Exercise 9.6. Give a term that is stuck and show which typing rule rejects it.

Solution. 3 4 (in LETREC, `(3 4)`) is not a value and no rule reduces it. T-App (Reduc) requires the operator to have a function type, and $3 : \mathbf{int}$, so the term is not well typed. The Rascal test `notProc` of `Letrec.rsc` checks this.

References

[1] Daniel P. Friedman and Mitchell Wand. *Essentials of Programming Languages.* The MIT Press, 3rd edition, 2008.

[2] Kent D. Lee. *Foundations of Programming Languages.* Springer, 2nd edition, 2017.

[3] David A. Watt. *Programming Language Design Concepts.* Wiley, 2004.